Linux Service Reliability Lab — Project Proposal

Author: Qiaowen Mei **Target role:** Backend / Platform Engineer **Stack:** Python, FastAPI, PostgreSQL, Redis, Nginx, Docker Compose, Prometheus, Grafana, Locust, Linux networking tools (tcpdump, ss, tc/netem, iptables) **Estimated duration:** 5–6 weeks part-time

---

1. Summary

This project builds a small but production-shaped backend service on Linux, then deliberately stresses it and breaks it. The goal is not to build a feature-rich app, but to produce measured, reproducible evidence of three backend skills hiring managers look for: designing a service that scales horizontally, tuning it with data rather than guesswork, and diagnosing failures quickly when they happen.

Every claim the project makes on a resume (concurrency, p95 latency, throughput gains, diagnosis time) will come from a benchmark or drill recorded in the repository, so each number can be defended in an interview.

2. Problem Statement

Most portfolio backends are tested by a developer clicking around on localhost. They never face concurrent load, slow dependencies, dropped packets, or a database that runs out of connections, which is exactly where real backend work gets difficult. As a result, candidates can describe caching or load balancing in theory but cannot say what happened to their p95 latency when they changed a worker count, or how they found the cause of a 502.

This lab closes that gap by treating the service as something to operate, not just write.

3. Goals and Non-Goals

Goals

The service should sustain at least 1,000 concurrent connections at under 120 ms p95 latency on the read-heavy workload. Tuning should produce a documented throughput gain and latency reduction, each attributable to a specific change. Injected failures should be diagnosable using dashboards, logs, and packet captures, with diagnosis time measured before and after observability work. Finally, anyone should be able to reproduce every benchmark with one command.

Non-goals

This project does not include authentication, multi-region deployment, Kubernetes, or a frontend. Those add scope without strengthening the core backend story.

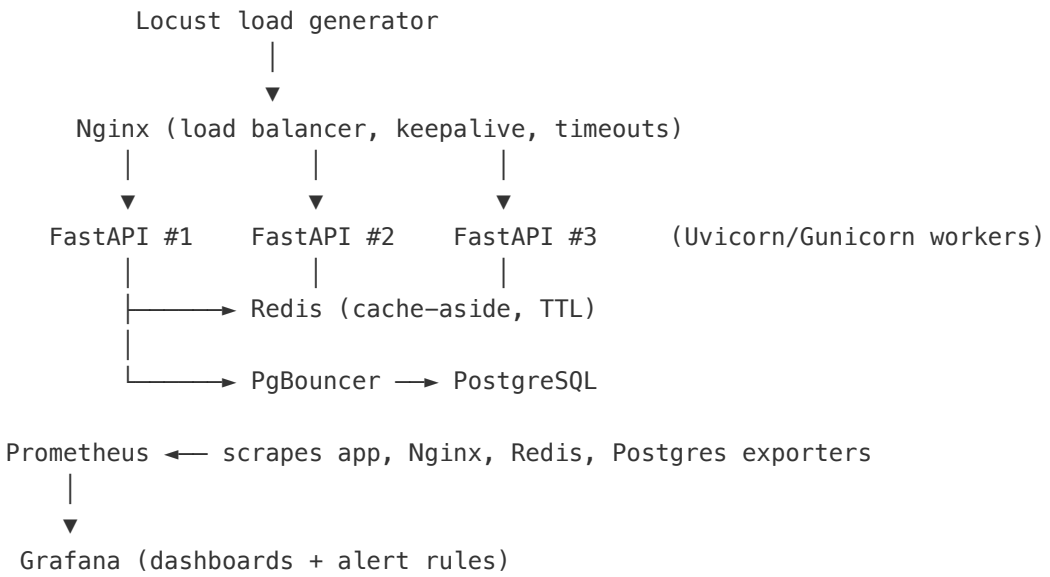
4. Service Design

The application is a simple URL-shortener-style API, chosen because it has a realistic read/write mix (reads vastly outnumber writes), natural caching opportunities, and clear correctness checks.

Endpoint	Purpose	Backend concern it exercises
`POST /links`	Create a short link	Writes, unique constraints, idempotency

`POST /links`	Create a short link	Writes, unique constraints, idempotency keys
`GET /links/{code}`	Resolve a link	Cache-aside reads, hot keys
`GET /links/{code}/stats`	Hit counts	Aggregation queries, write amplification
`GET /health/live`, `/health/ready`	Liveness and readiness	Load balancer health checks, dependency checks
`GET /metrics`	Prometheus scrape	Instrumentation

5. Architecture



Everything runs in Docker Compose on a single Linux host (a cloud VM is preferred over a laptop so load generator and service can be separated onto different machines for cleaner numbers).

Key backend decisions

Stateless app replicas behind Nginx. Any replica can serve any request, which allows horizontal scaling and lets the lab test what happens when one replica dies mid-load.

Cache-aside with Redis. Reads check Redis first, fall back to PostgreSQL on a miss, and populate the cache with a TTL. Writes invalidate the key. This exposes real tradeoffs around staleness, cache stampedes, and hit rate.

Connection pooling with PgBouncer. Without pooling, each worker opens its own database connections, and PostgreSQL's connection limit becomes the bottleneck. Measuring before and after pooling is one of the core tuning experiments.

Idempotent writes. `POST /links` accepts an `Idempotency-Key` header so client retries during timeouts do not create duplicates.

Graceful shutdown and readiness. Replicas fail readiness checks before shutting down, so Nginx drains them without dropping in-flight requests.

6. Benchmarking Plan

All load tests use Locust with a fixed, versioned workload profile: 90% reads with a skewed (Zipfian) key distribution to create realistic hot keys, 8% writes, and 2% stats queries. Each test runs a warm-up period, then a fixed measurement window, and is repeated three times to report the median.

Each experiment changes exactly one variable and records throughput, p50/p95/p99 latency, error rate, cache hit rate, and database connection count.

Experiment	Variable changed	Expected insight
Baseline	Single replica, no cache, no pooling	Reference numbers
Worker concurrency	Uvicorn workers per replica (1, 2, 4, 8)	Where CPU saturates vs. where workers just contend
Horizontal scaling	Replicas behind Nginx (1, 2, 3)	Whether throughput scales linearly
Caching	Redis off vs. on, varying TTL	Hit rate vs. latency vs. staleness
Connection pooling	Direct connections vs. PgBouncer	Removing the database connection bottleneck
Nginx tuning	Upstream keepalive, timeouts	Effect of connection reuse on tail latency
Linux limits	`ulimit -n`, `net.core.somaxconn`	Finding and raising OS-level connection ceilings

The throughput and latency improvements reported on the resume will be the measured difference between the baseline and the final tuned configuration, with each contributing change documented.

7. Failure Injection and Diagnosis

The second half of the project treats the service as an on-call engineer would. A script injects one failure at a time, and the diagnosis is timed from the moment symptoms appear to the moment the root cause is confirmed.

Failure	How it is injected	Primary diagnosis tool
Network latency to the database	`tc qdisc add ... netem delay 200ms`	Grafana latency panels, query timings
Packet loss	`tc ... netem loss 10%`	tcpdump retransmissions, error rate
Redis outage	Stop the Redis container	Cache hit rate drop, fallback latency
Database connection exhaustion	Lower the PostgreSQL connection limit	PgBouncer and Postgres connection metrics
App replica crash	Kill a container under load	Nginx upstream errors, readiness checks

Blocked port	`iptables` DROP rule	`ss` , tcpdump, health checks

Each failure is run twice. The first round uses only raw container logs. The second round uses the full observability stack: Grafana dashboards, structured JSON logs with request IDs, health checks, and alert rules. The difference in average diagnosis time between the rounds becomes the measured observability impact.

Every drill produces a short incident write-up covering symptoms, timeline, root cause, and fix, stored in the repository as evidence.

8. Observability

The app exposes request count, latency histograms, and error rates per endpoint, plus cache hit/miss counters. Exporters cover Nginx, Redis, and PostgreSQL. Grafana dashboards follow the RED method (rate, errors, duration) for the service and the USE method (utilization, saturation, errors) for dependencies. Alert rules fire on high p95 latency, elevated error rate, replica unavailability, and database connection saturation.

9. Timeline

Week	Milestone
1	FastAPI service, PostgreSQL schema, Docker Compose, health endpoints
2	Nginx load balancing, Redis cache-aside, PgBouncer, idempotent writes
3	Prometheus, exporters, Grafana dashboards, structured logging
4	Locust workload profile, baseline benchmarks, tuning experiments
5	Failure injection scripts, timed diagnosis drills, incident write-ups
6	Final benchmark report, README, architecture diagram, cleanup

10. Deliverables

The repository will contain the full source and Docker Compose setup, a one-command benchmark runner, versioned Locust workloads, a benchmark report with tables and charts comparing every experiment, failure injection scripts with incident write-ups, and exported Grafana dashboards.

11. Success Criteria and Resume Mapping

Resume claim	How it will be measured
1,000+ concurrent connections at sub-120 ms p95	Final configuration Locust run, median of 3
Throughput gain and p95 reduction from tuning	Baseline vs. tuned configuration, with each change attributed
Cache effectiveness	Measured hit rate under the Zipfian workload

Faster incident diagnosis	Average drill time, logs-only round vs. full observability round

12. Risks and Mitigations

Unrealistic numbers from a single machine. Running the load generator on the same host as the service distorts results. The mitigation is to run Locust from a separate VM and state the test environment clearly in the report.

Scope creep. Adding features to the app dilutes the reliability story. The API stays at five endpoints, and new work goes into experiments rather than features.

Biased diagnosis timing. Diagnosing the same failure twice makes the second attempt faster through familiarity alone. The mitigation is to randomize the failure order and, where possible, have a friend inject failures blind.

13. Interview Value

This project gives concrete answers to common backend interview questions: how to find a bottleneck (the experiment table), how caching changes latency and consistency (the TTL and invalidation results)